

nwctl 使用手册

nwctl 使用手册

命令速查表

用户管理

命令	说明
nwctl register <name> --src <path>	注册新用户，绑定源码目录
nwctl register <name> --src <path> --domain-id <0-232>	使用指定的可用 ROS Domain ID 注册
nwctl update <name> --src <path>	更新用户源码路径
nwctl unregister <name>	注销用户并删除 build/install/log workspace
nwctl unregister <name> --keep-workspace	注销用户（明确保留 workspace）
nwctl list	列出所有已注册用户及其 DOMAIN_ID

运行模式

命令	说明
nwctl <name> planning-sim	启动规划仿真（使用镜像预编译版本）
nwctl <name> rosbag-replay	启动 rosbag 回放（使用镜像预编译版本）
nwctl <name> shell	进入开发 shell（挂载用户源码）
nwctl <name> stop	停止该用户所有运行中的容器
nwctl <name> clean	清理该用户的 build 缓存

状态与运维

命令	说明
nwctl status	显示所有正在运行的 aw-* 容器
nwctl disk	显示每个用户的磁盘占用（workspace）
nwctl cleanup	清理孤立容器和临时文件
nwctl cleanup --dry-run	预览 cleanup 会清理什么（不实际执行）
nwctl check-env [mode]	检查运行环境（Docker/GPU/镜像/Display/数据/资源）
nwctl version	显示版本号
nwctl pull	拉取或更新配置的容器镜像
nwctl pull --profile cpu	拉取无 GPU 的 Humble 开发镜像
nwctl completion <bash\ zsh>	输出当前源码目录的补全启用命令
nwctl -h	显示帮助

CPU 与 NVIDIA Profile

默认 auto 会检测 NVIDIA GPU 和 Docker runtime：两者都可用时使用 nvidia，否则使用 cpu。

Profile	镜像	运行参数
cpu	ghcr.io/autowarefoundation/autoware:universe-devel-humble	软件渲染，不传 NVIDIA runtime
nvidia	ghcr.io/autowarefoundation/autoware:universe-devel-cuda	--runtime=nvidia

```
nwctl check-env shell --profile cpu
nwctl <name> shell --profile cpu
nwctl <name> planning-sim --profile cpu
```

可设置 NWCTL_PROFILE=cpu，让当前主机默认使用 CPU 模式。

三种运行模式详解

1. planning-sim — 规划仿真

nwctl <name> planning-sim

- 使用镜像内 /opt/autoware（预编译），无需本地编译
- 自动打开 rviz2 窗口，通过 X11 转发显示

操作步骤：

1. 等待 rviz2 窗口出现（约 30 秒）
2. 点击工具栏 **2D Pose Estimate**，在地图上点击拖拽设置初始位姿
3. 点击 **2D Goal Pose** 设置目标点
4. 在 **AutowareStatePanel** 面板中点击按钮启动自动驾驶

2. rosbag-replay — 数据包回放

nwctl <name> rosbag-replay
nwctl <name> rosbag-replay --bag <子目录> --rate 0.5

- 使用镜像内预编译版本
- 自动启动 Autoware，等待初始化后执行 ros2 bag play
- --bag 选择 --rosbag-path 下的子目录，--rate 设置回放速度
- CPU 模式默认关闭依赖 CUDA/ML 模型的感知管线，并等待 60 秒后开始回放；地图、传感器解码、定位和 rviz 保持启用
- 可通过 NWCTL_ROSBAG_START_DELAY 调整自动回放前的等待时间
- 在 rviz Views 面板中将 **Target Frame** 设为 base_link 以跟随车辆

3. shell — 开发模式

nwctl <name> shell

- 将用户源码目录挂载到容器 /workspace/src
- 容器内叠加 /opt/autoware（预编译）作为底层依赖

容器内常用操作：

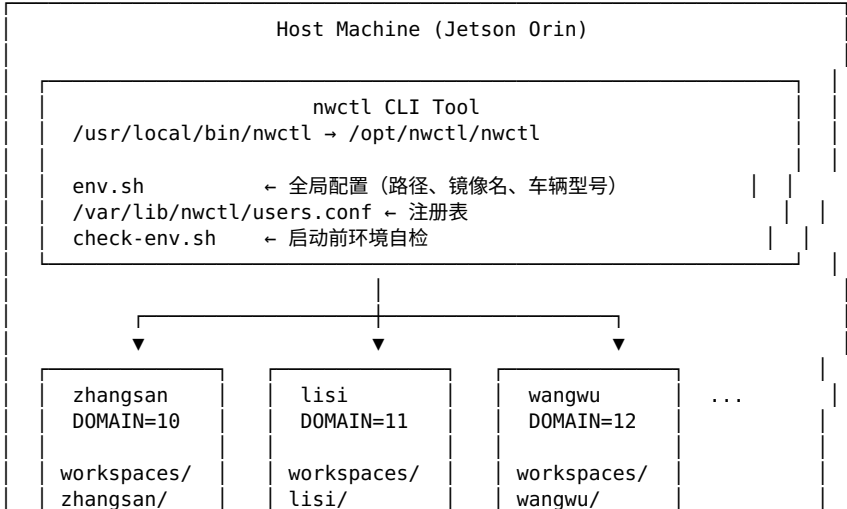
```
# 增量编译单个包（最常用）
colcon build --packages-select <pkg> --symlink-install

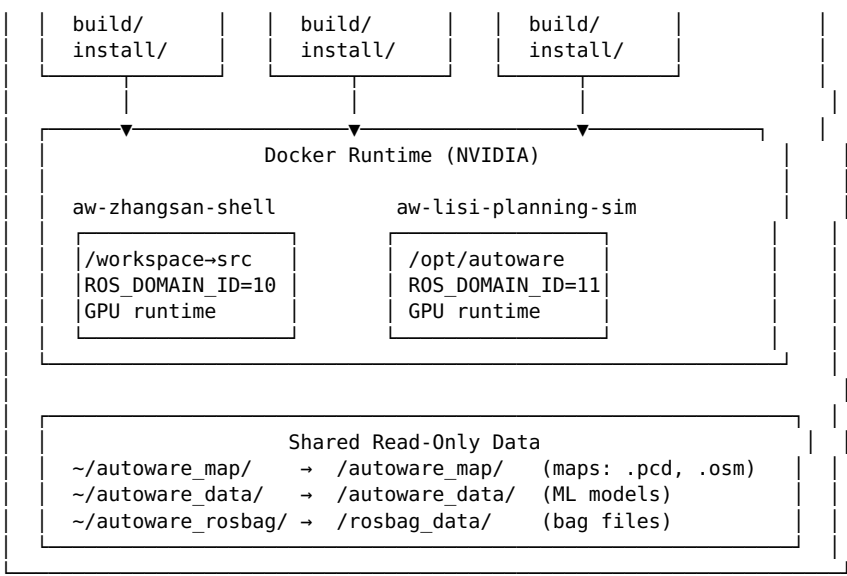
# 编译包及其所有依赖
colcon build --packages-up-to <pkg> --symlink-install

# 全量编译（首次，耗时较长）
colcon build --symlink-install --cmake-args -DCMAKE_BUILD_TYPE=Release

# 重新 source 后启动仿真测试
source /workspace/install/setup.bash
ros2 launch autoware_launch planning_simulator.launch.xml \
  map_path:=/autoware_map/sample-map-rosbag \
  vehicle_model:=sample_vehicle \
  sensor_model:=sample_sensor_kit
```

架构图





容器模式对比

模式	源码来源	setup.bash 路径	适用场景
planning-sim	镜像预编译	/opt/autoware/setup.bash	测试、演示、验证
rosbag-replay	镜像预编译	/opt/autoware/setup.bash	数据回放分析
shell	用户挂载源码	/workspace/install/setup.bash	开发、调试、修改算法

隔离机制

资源	隔离方式	说明
ROS 话题	ROS_DOMAIN_ID	每用户自动分配，从 10 开始，跳过保留 ID（5）
源代码	独立 git clone	每人管理自己的仓库和分支
编译产物	独立挂载	/var/lib/nwctl/workspaces/<user>/build, install, log
容器命名	aw-<user>-<mode>	避免冲突，便于管理
注册表写入	flock（10s 超时）	users.conf.lock 防止并发写冲突
GPU	共享	NVIDIA runtime 支持多容器同时使用

典型工作流程

管理员（一次性）

```
# 拉取 Docker 镜像
docker pull ghcr.io/autowarefoundation/autoware:universe-devel-cuda

# 安装 nwctl
cd ~/autoware/nwctl
sudo ./install.sh
# Bash/Zsh 补全会同时安装，重新打开终端后按 Tab 使用
```

团队成员（每人）

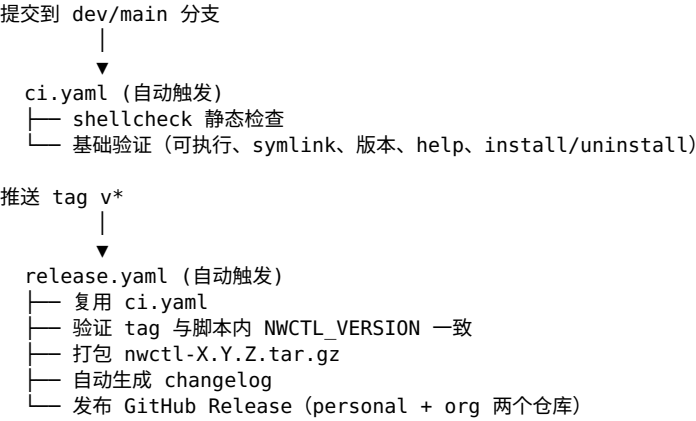
```
# 1. 克隆自己的代码
cd ~
git clone https://github.com/autowarefoundation/autoware.git myname_aw
cd myname_aw
vcs import src < repositories/autoware.repos

# 2. 注册
nwctl register myname --src ~/myname_aw/src

# 3. 环境自检
nwctl check-env

# 4. 按需使用
nwctl myname planning-sim      # 测试
nwctl myname rosbag-replay    # 回放
nwctl myname shell              # 开发
```

CI/CD 流程



远端仓库：

别名	地址	用途
origin	git@github.com:LiZheng1997/nwctl.git	个人仓库
org	git@github.com:uwa-rev/nwctl.git	组织仓库

FAQ

Q: rviz2 黑屏或崩溃

```
xhost +local:docker
echo $DISPLAY # 应为 :0 或 :1
```

Q: GPU 错误 “could not select device driver”

```
docker run --rm --runtime=nvidia nvidia/cuda:12.0.0-base-ubuntu22.04 nvidia-smi
# 失败则重装 nvidia-container-toolkit 并重启 Docker
```

Q: 多用户 ROS 话题互相干扰

每个用户自动分配唯一 ROS_DOMAIN_ID，设计上完全隔离，无需手动处理。

Q: colcon build 找不到依赖

```
source /opt/autoware/setup.bash # 先 source 预编译版本
colcon build --packages-select <pkg> --symlink-install
```

Q: 点云地图权限错误

```
sudo chmod 644 ~/autoware_map/sample-map-rosbag/pointcloud_map.pcd
# 或使用 check-env 自动检测
nwctl check-env
```

Q: 编译产物在容器重启后是否保留？

保留。安装版的编译产物位于宿主机 /var/lib/nwctl/workspaces/<user>/ 目录中，容器重启不影响。